Zurich University of Applied Sciences
School of Engineering
InES Institute of Embedded Systems

TSM_EmbHardw: Scheduling

Index

1 Aims

In this lecture we learn some of the more common scheduling policies that are common to both HW and SW design. This list incudes

1. List Scheduling
2. First In First Out
3. Round Robin
4. Rate Monotonic
5. Voltage Scheduling

The student will be able to explain the use and name examples for that use, of all the schedulers covered in this lecture.

2 Lecture Prologue

Scheduling is based on queuing theory which in itself originated from factory-floor workshops. Production because there are, for example, only so many drilling machines on a factory floor and typically more jobs/tasks than there are machines, i.e. queues. In many cases these jobs/tasks have differing priorities so sort of scheduling must be applied. In other words as soon as there are more requests for resources than the resource itself, scheduling must be applied. Hennesy and Patterson discuss various forms and application domains of scheduling used in both HW and SW.

3 Lecture

Back To Index

In this lecture we look at:

1. List Scheduling
2. Prefetching
3. Tiling
4. Locking
5. Tightly Coupled Memory

as well as some relevant case studies

3.1 Notes to List Scheduling

List scheduling is tightly bound to both HW design and compiler design. In particular the concept of effective use of HW pipelines (as opposed to instruction pipelines) demands scheduling as the example in the lecture shows. The use of list scheduling in this example is predicated on a number of situative facts.

1. We cannot assume our hardware features unlimited number of functional units
2. A schedule is required to allow the two independent calculations to proceed.
3. A schedule is required to determine the order of calculation for the first operations of the first equation

In terms of reduced number of functional units, which is especially prescient in FPGA design:

- The function executed as-is would require four multipliers, one adder, one subtractor and one comparator
- The function, as postulated in the example, is **allocated** 2 multipliers and two general purpose ALUs (to which the additions, subtraction and comparison are/is **bound**)
- This allocation implies optimisation by parallelisation
- This allocation is however not enough to execute the function without additional design effort
- This design effort is pipelining
- Pipelining (as explained in [EMBHW]) involves separating functional stages by clocked registers.
- Therefore, a simplistic schedule with a degree of parallelisation is possible
- In the example given, this schedule can be improved by list scheduling
- The sequence of operations is determined by those with the maximum number of following operations, first.
- Once the ideal schedule has been arrived at, the execution time can be improved by slack-minimisation, which is a pipelining optimisation.

In the example shown below

- there is no parallelisation possibility (far left (a))
- there is no scope for a schedule other than that de-facto defined by the equation
- there is potential for pipelining to reduce the number of required functional units (middle (b))
- there is (realised) potential for slack minimisation (far right (c))

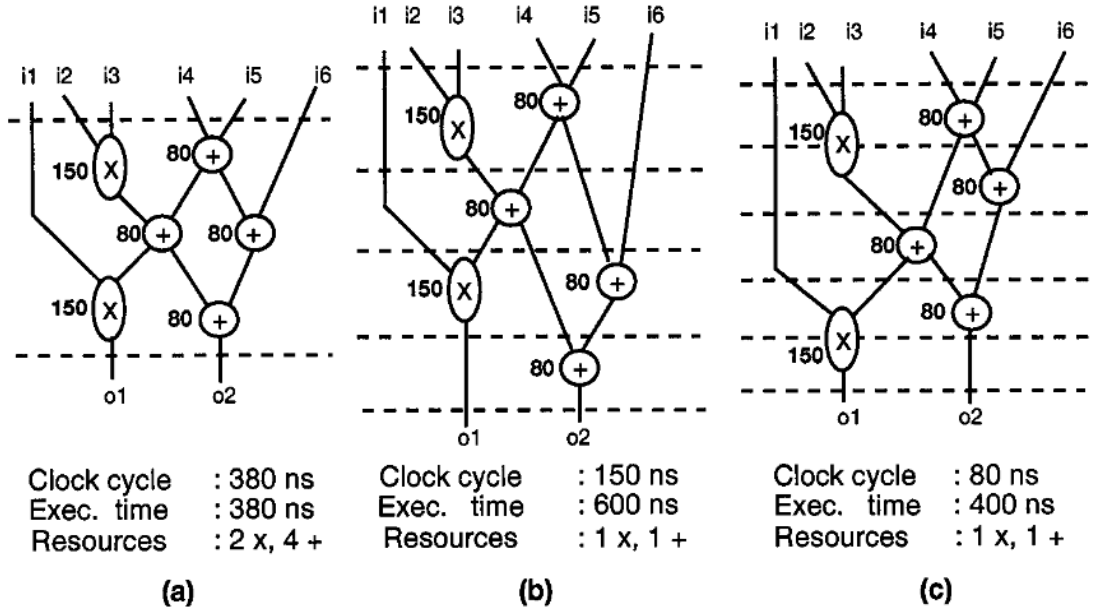

Fig. 1. Effect of clock period on execution time and resources required.

Figure 1: Depiction of a simple pipline system performing a mathematical operation

From: [Chang]

3.2 Notes to FIFO Scheduling

The scheduler maintains a queue called the **ready queue**. So called because all tasks in the queue are "ready" for execution. This terminology is further formalised in the state models for tasks. The popular theoretical one is the 7-state model, practical ones include the 4 state model from Linux and the two state model from eCOS [eCOS]. See [[Tannenbaum]] for further details.

3.3 Notes to Round Robin Scheduling

To compare FIFO and RR scheduling

- RR scheduling necessitates **preemption**, the tasks must be interruptable
- FIFO is **run to completion** and is **non-premptive**
- The **turnaround time** (completion time - arrival time) of task P5 is 12 time units using a FIFO scheduler
- The turnaround time of task P5 is 7 time units using a RR scheduler
- **Fairness** is encouraged using RR scheduling against FIFO scheduling
- In the example T_2 arrives at the back of the queue at the same time as T_1. Priority is given to the newly arrived task.

3.4 Notes to Rate Monotonic

- RR and FIFO, as presented, do not support priorities
- Under Rate Monotonic, the de-facto priority is given to the task with the highest repetition rate
- RM is reflective of the fact that most RT tasks are repetitive tasks executed at constant time intervals
- A system with a lot of slack represents poor utilisation of computing resources
- Such a system can be called **over-provisioned** and represents unnecessary expense (cost)
- RM interesting because the utilisation of a CPU can be calculated a priori
- Nevertheless, this predictability comes at a cost of ~30% slack which is quite conservative.

- Hence, the predictability of "pure" RM is of limited use in practice.
- RTEMS [RTEMS] supports an RM manager
- Further reading - see [Shen] on how to use resources when execution time < WCET

3.5 Notes to Voltage Scheduling

- Example taken from [Luo]
- Voltage scheduling attempts to decrease the required power by decreasing the operating clock.
- This has the effect of increasing latency, the task is to determine by how much
- The bounds are given, in this case, by deadlines
- A secondary effect is also to increase utilisation
- The utilisation bounds are given by the cost categories of slower processors

---

4 Lecture Epilogue

Back To Index

4.1 List Scheduling & Pipelining

Further reading: on slack minimisation for asynchronous systems [Gill] on power and pipelining [Chinnery]

4.2 FIFO Scheduling

(This section intentionally left blank)

4.3 RR Scheduling

Early operating systems (UNIX, Linux V1) and some embedded OS used priority RR schemes called **multilevel scheduler**. Simplistic use of these leads to an effect called **starvation**, tasks of lowest priority never get scheduled time. This can be mitigated by a scheme called **multilevel feedback** which involves increasing the time quanta granted. See [Tannenbaum] for further details.

When speaking of CPU- and I/O bound tasks. I/O bound tasks spend much of their time waiting for I/O in a **blocked queue**. In the process of being transferred from the ready to the blocked queue, the I/O tasks gives up its time quanta. This means that the tasks at the head of the ready queue is dispatched immediately. This task does not inherit the remaining quanta, it only runs for its previously allocated time quanta.

4.4 Rate Monotonic Scheduling

(This section intentionally left blank)

4.5 Voltage Scheduling

- Voltage scheduling is only one example of a power reduction function
- Putting processor parts, or the processor itself, to sleep (e.g. A/D converters) for a period of time is also common e.g. [ESP]
- For many applications - communication for instance - there is no scope to reduce the voltage/frequency

- In these cases communication processors can be simply turned off until required, typical in the IoT domain - e.g. [NORD]
- ARM [ARM-1] have been continuously developing their big.LITTLE architecture for a decade or so
- ARM [ARM-2] also support DVS
- [Yokoyama] on accounting for battery characteristics when using DVS

---

5 Exercises

Back to Index

Exercise 1 - Queuing

An co-processor exclusively executing edge detection services the input from three cameras. The edge detection calculation has a worst case execution time of 250 ms. A driver, maintaining a queue, receives calculation requests isochronously from three cameras on a FIFO basis. What is the max rate of requests if the queue can only hold three pending requests?

Answer

Exercise 2 - Schedules FIFO and RR

Consider the following task table - what is the schedule under FIFO and RR? Remember - if two tasks enter the queue at the same time, the newer task has priority

Task	Arrival Time	Burst Time (WCET)
T_1	0	4
T_2	0	3
T_3	0	5

Answer

Exercise 3 - Rate Monotonic Scheduler and Voltage Scheduling

An embedded system features a single processor with three recurring tasks to be performed with the following repetition times

Task	Repetition	Burst Time (WCET)
T_1	1	0.3
T_2	1	0.3
T_3	2	0.4

1. Which task has the highest priority under a rate monotonic scheduler?
2. Is there a guaranteed schedule? Support your answer with the relevant calculation
3. Is there a schedule?

4. Determine the schedule
5. Is this system a candidate for voltage scheduling and if so calculate the task reduction ratio
6. Would there be anything to be gained by using a Round Robin scheduler rather than a rate monotonic? Explain

Answer

Exercise 4 - RM & Voltage Scheduling

Consider the following task table and answer the following questions

Task	Repetition	Burst Time (WCET)
T_1	10	4
T_2	15	3
T_3	5	1

1. Which task has the highest priority
2. Is there a guaranteed schedule?
3. Is there a schedule?
4. How much slack does the system have?
5. Is the system well provisioned?
6. Is the system a candidate for voltage scheduling?

Answer

Exercise 5 - Pipelining and List Scheduling

Consider the following equation

```
(( A*B ) * ( B+C ) + (( C*D* E ) + ( C*D )
```

Given one multiplier with 4 time units (TUs) WCET and one adder with 2 TUs WCET

1. Draw the data flow graph
2. Schedule the operations using list scheduling
3. Pipeline the design
4. How much slack is there in the system if it is clocked according to maximum operator delay?
5. How much slack is there if it is clocked using slack minimisation?
6. Repeat 2 .. 5 if you are given two multipliers
7. What factors would you need to consider when making a decision wither to expend the additional multiplier?
8. When must variable E be ready?

Answer

6 Laboratory

(This section intentionally left blank)

Back To Index

Glossary

(This section intentionally left blank)

References

EMBHW TSMEmbHardw_MAC_2_GPU.pdf Course materials TSM_EmbHardw.

Chang En-Shou Chang, Daniel D. Gajski, and Sanjiv Narayan. 1996. An optimal clock period selection method based on slack minimization criteria. ACM Trans. Des. Autom. Electron. Syst. 1, 3 (July 1996), 352–370. https://doi.org/10.1145/234860.234864

Hennessy and Patterson Hennessy, John L.; Patterson, David A.. Computer Architecture: A Quantitative Approach (The Morgan Kaufmann Series in Computer Architecture and Design) (p. 292). Elsevier Science. Kindle Edition.

Luo Jiong Luo, N. Jha 'Static and dynamic variable voltage scheduling algorithms for real-time heterogeneous distributed embedded systems' Proceedings of ASP-DAC/VLSI Design 2002. 7th Asia and South Pacific Design Automation Conference and 15h International Conference on VLSI Design http://dx.doi.org/10.1109/ASPDAC.2002.995019,

Shen C. Shen; K. Ramamritham; J.A. Stankovic 'Resource reclaiming in multiprocessor real-time systems' IEEE Transactions on Parallel and Distributed Systems (Volume: 4, Issue: 4, April 1993) https://doi.org/10.1109/71.219754

Yokoyama Tetsuo Yokoyama; Gang Zeng; Hiroyuki Tomiyama; Hiroaki Takada 'Heuristics for Static Voltage Scheduling Algorithms on Battery-Powered DVS Systems' 2009 International Conference on Embedded Software and Systems https://doi.org/10.1109/ICESS.2009.19

Gill G. Gill, V. Gupta and M. Singh, "Performance estimation and slack matching for pipelined asynchronous architectures with choice," 2008 IEEE/ACM International Conference on Computer-Aided Design, San Jose, CA, USA, 2008, pp. 449-456, doi: 10.1109/ICCAD.2008.4681614.

Chinnery Chinnery, D., Keutzer, K. (2007). Pipelining to Reduce the Power. In: Closing the Power Gap Between ASIC & Custom. Springer, Boston, MA. https://doi.org/10.1007/978-0-387-68953-1_3

Stallings William Stallings 'Operating Systems: Internals and Design Principles' 9th Edition, Pearson ISBN-13: 9780137516742 (2021 update)

Tannenbaum Andrew Tannenbaum 'Modern Operating Systems' 4th Edition, Pearson, 2014

ARM-1 'big.LITTLE' https://developer.arm.com/documentation/den0013/0400/big-LITTLE Last accessed 21.10.2025

ARM-1 'big.LITTLE' https://developer.arm.com/documentation/den0013/0400/Power-Management/Dynamic-Voltage-and-Frequency-Scaling?lang=en Last accessed 21.10.2025

RTEMS '12. Rate Monotonic Manager' https://docs.rtems.org/docs/main/c-user/rate-monotonic/index.html Last accessed 21.10.2025

eCOS 'eCOS Overview' https://www.ecoscentric.com/ecos/index.shtml Last accessed 21.10.2025

NORD-1 'nRF5340' https://www.nordicsemi.com/Products/nRF5340 Last accessed 21.10.2025

ESP-1 'Sleep Modes' https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/sleep_modes.html Last accessed 21.10.2025

Answers to exercise questions

Back To Index

Answer to exercise 1 - Queuing

Entry into the queue permissible every 250 ms. If the cameras supply requests at an equal rate that is one request every 750 ms for each camera.

Answer to exercise 2 - Schedulers FIFO and RR

Scheduler	Schedule
FIFO	$T_1T_1T_1T_1T_2T_2T_2T_3T_3T_3T_3T_3$
RR	$T_1T_2T_1T_3T_2T_1T_3T_2T_1T_3T_3T_3$

Answer to exercise 3 - Rate Monotonic & Voltage Scheduling

1. T1 and T2 have equal priority
2. 300/1000 + 300/1000 + 400/2000 = 800/1000 = 75% utilisation no
3. yes
4. Yes, there is a slack of 550ms which can be used to reduce energy consumption
5. 1000 – 0 – (800) / 800 + 1 = 1.25
6. Difficult to see – The overhead would be not worth it – RR is dynamic RM is a priori

Answer to exercise 4 - Rate Monotonic

1. (T_3)

2. No - the tasks have a utilisation of 80%, for a guaranteed scheduled the utilisation must be ~77% or under

3. Yes, see below - each slot represents 5 time units (TU)

Task	Slot 1	Slot 2	Slot 3	Slot 4	Slot 5	Slot 6	Slot 7	Slot 8	Slot 9	Slot 10
T_1	1	3	0	4	0	0	1	3	0	4

Task	Slot 1	Slot 2	Slot 3	Slot 4	Slot 5	Slot 6	Slot 7	Slot 8	Slot 9	Slot 10
T_2	3	0	3	0	3	0	3	0	3	0
T_3	1	1	1	1	1	1	1	1	1	1
Total TUs	5	4	4	5	5	4	1	4	4	5

4. 20%

5. Matter of opinion: if you assume that this is an under-estimation then 15%-10% is acceptable and possibly not worth a re-design to cheaper hardware

6. Not really, each individual time slot would require separate reduction and the management effort would probably outweigh any benefits

Answer to exercise 5 - Pipelining and List Scheduling

1.

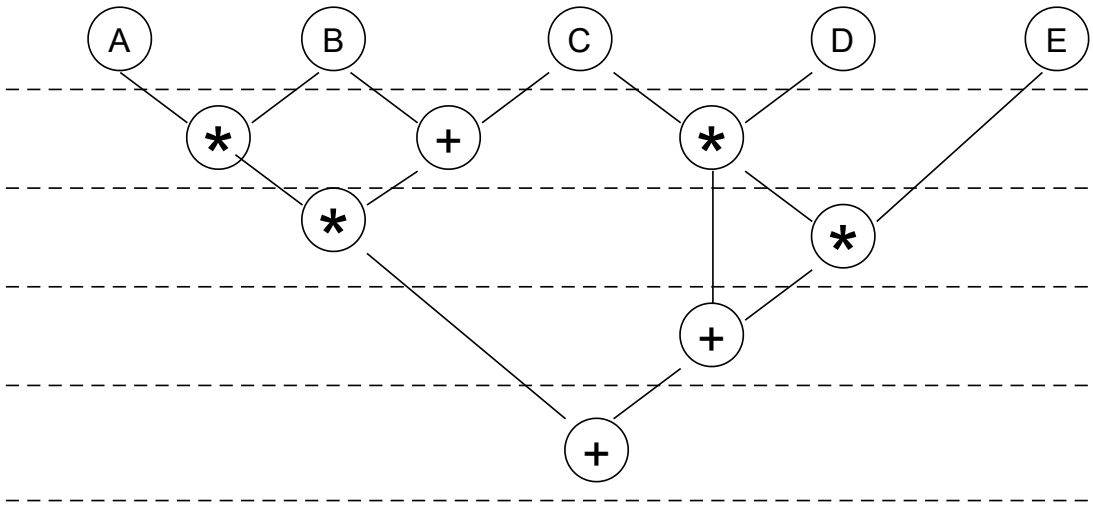

Figure 2: Dataflow of exercise 5

2. & 3. -> see above for the path with the maximum number of dependencies

Operator	Pipeline Stage 1	Pipeline Stage 2	Pipeline Stage 3	Pipeline Stage 4	Pipeline Stage 5	Pipeline Stage 6
Multiplier	C * D	(C * D) * E	A * B	(A * B) * (B + C)		
Adder	B + C		(C * D) + ((C * D) * E)		final addition	

4. The clock is 4 TUs (maximum operator delay) so the adder path exhibits in total 4 TUs slack. The final addition is dependent on the completion of the multiplication.

5. By clocking the pipeline at 2 TUs (fastest common clock), 2 TUs can be shaved off the total operation time. The dependency of the final addition on the final multiplication precludes further total operation optimisation.

Operator	**Pipeline Stage 1**	**Pipeline Stage 2**	**Pipeline Stage 3**	**Pipeline Stage 4**	**Pipeline Stage 5**	**Pipeline Stage 6**
Multiplier 1	C * D	(C * D) * E				
Multiplier 2	A * B	(A * B) * (B + C)				
Adder	B + C		(C * D) + ((C * D) * E)	final addition		

6. The slack calculations don't change much

The following points should be considered - the killer criteria is in bold

1. (latency) The parallelisation using two multipliers reduces the number of pipeline stages by one (or two if slack minimisation is applied)
2. (latency) The total optimisation is 20%.
3. (latency) **Is the additional speed necessary**?
4. (cost) The multiplier has already been designed and the addition of a second in the (system) design is trivial so it's a footprint-only consideration
5. (power) The peak power due to the additional multiplier doubles but the average stays the same

HW-SW co-design and design space optimisation (formally) seeks to optimise a cost-function on a necessary-only basis. If the latency requirements do not require the additional TUs then no - the second multiplier should not be added (premature optimisation).
